# Model Predictive Control System with Single-Shot Object Recognition for Automobile Safety

## Statement of Problem

Entering the new age of AI technology, **self-autonomous vehicles are becoming more and more popular to the public**. Companies such as Tesla and Google are using AI in autonomous vehicles to avoid collisions and obstacles, however there is a major issue: unpredictability of the vehicle during an incident. Since 2022, there have been **396 major car accidents involving self-autonomous cars**, with this number rising each year. This project proposes a **solution to movement unpredictability** by using local odometry tracking, Dynamic Window Approach (DWA) path planning, Model Predictive Control (MPC) path following, and Single-Shot Detection (SSD) AI object recognition. The synergy of these algorithms and functions allows for safer vehicles to be on the road and will greatly help the elderly and those who have disabilities.

## Introduction

Fully autonomous vehicles use technology to entirely replace the human driver in navigating a vehicle, avoiding road hazards and responding to traffic conditions through its various sensors and systems. Currently, these **fully autonomous vehicles are not completely reliable**, with many accidents having been reported due to lack of reliable decision-making systems. The proposed project will utilize a local pathfinding and decision-making system rather than a global system, giving it faster reaction times to unexpected obstacles and better control in these abrupt situations. It involves a dynamic path-planning system that continuously updates its route using AI and sensor data**, ensuring safer and more efficient navigation.**

**The projects goals are as follows:**

- **Build a VEX model vehicle** to test Model Predictive Control (MPC) path following and an SSD object recognition AI model using a DWA path planner
- Code an **MPC system** that follows a desired path with low variability and adapts to the environment without making unsafe maneuvers
- Code a **Single-Shot Detection object recognition AI model** that precisely identifies objects, their sizes, and their location local to the robot
- Code a **Dynamic Window Approach (DWA) path planner** that creates a desired path around an obstacle for the MPC path following algorithm to follow
- **Test and gather movement data** to ensure the safety of this technology

### Projected Number of Electric Vehicles Based on Previous Data (2019-2024)

Quadratic Regression Model: $y = 0.61x^2 + -2463.259x + 2486752.3982$

As shown in the Year vs. Number of Autonomous Vehicles (Fig 1.1), there will be around **83,000** autonomous vehicles in use by 2030, almost **250% more** than in 2025.

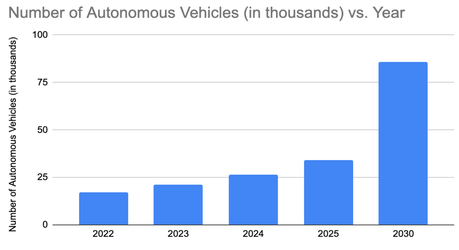



Figure 1.1. ADAS Accident Prediction using Quadratic Regression
Source: statista.com

The coefficient of determination, ($r^2 = 0.99463$) for the quadratic model determines that **99.4%** of variation in the number of autonomous vehicles can be explained by an approximate linear relationship with the year. Based upon this predicted number of autonomous vehicles, assuming a constant ratio of accidents to total number of vehicles, we can assume around **2,500 crashes to occur in 2025**.

## Materials

**VEX Robot Car Model**

- Bambu Labs P1S 3-D Printer
- Translucent PETG Printing Filament
- x6 36 Tooth Plastic Gears
- x8 60 Tooth Plastic Gears
- x8 Stainless Steel Axles
- Nylon Spacers
- 1x2x1 Aluminum C-Channel
- x8 3.25" Traction Wheels
- 24"x16" 1/16" Black Delrin Sheet
- 60W OMTech Laser Cutter
- x1 VEX V5 Brain
- x1 VEX Radio
- 11W 600RPM VEX V5 Motors
- #8 x ½-inch 8-32 Stainless Steel Torx Screws
- 8-32 Stainless Steel Nylon Insert Lock Nuts
- Grizzly 15" Floor Drill Press
- Grizzly Industrial 1hp Belt Sander
- 4300 Dremel 35000 RPM Hand Drill

**Computer-Aided Design (CAD)**

- OnShape CAD Software
- VEX V5 Parts Library

**Pathing Algorithms: MPC, DWA, Odometry Tracking**

- Visual Studio Code (C++)
- PROS CLI 3.5.4
- PROS Kernel 4.1.1
- MATLAB
- QT Creator 15.0.0
- DESMOS Graphing Calculator
- Microsoft Excel
- LaTeX Overleaf

**SSD Object Recognition AI Model**

- PyCharm (Python)
- TensorFlow Library
- Common Objects in Context Dataset
- Intel D435i Depth Camera
- Raspberry Pi 4 Model B
- Micro USB Cable
- RTX 4070 GPU

## Methods

**3-D Design**

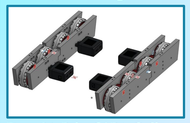


Figure 2.1. Robot CAD in OnShape
Source: Author

The first step in the engineering process is to CAD the robot in OnShape (Fig 2.1) where electronics and structure is designed.

**Build**

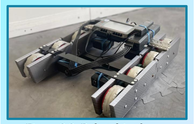


Figure 2.2. Robot drive base
Source: Author

The vehicle model was built for testing (Fig 2.2) and was ready to imitate a real vehicle but still needed software to operate autonomously.

**Code**

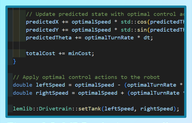



Figure 2.3. MPC velocity controller
Source: Author

Algorithms and the AI for the robot were coded (Fig 2.3), which include MPC, pure pursuit, odometry, DWA, and SSD Object Recognition.

**Test and Analyze**

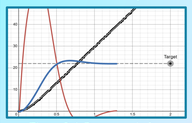



Figure 2.4. Pure pursuit data
Source: Author

Each algorithm was tested individually, and error data was gathered then visualized to ensure the robot was precise (Fig 2.4).

### 3-D Printing Robot Chassis

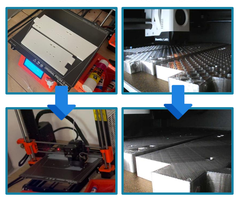



Figure 3.1. 3-D printed structure for chassis
Source: Author

Due to warping issues from rapid temperature changes, the printer changed from a Prusa i3 to a Bambu Labs P1S. Moreover, the plastic changed from PLA to PETG for additional durability (Fig 3.1).



Figure 4.1. Robot Structure
Source: Author



Figure 4.2. Traction Wheels
Source: Author

### Hands-On Drive Base Build

The VEX Robot model, for simulating an actual car, was assembled with a 3D-printed structure, Delrin plastic, an aluminum bar, traction wheels, and much more (Fig 4.1, 4.2, 4.3, 4.4). The robot was designed such that its speed and size were in **proportion to an actual car** so the tested algorithms could be applied in actual cars. After building the robot, electronics were then mounted including the Raspberry Pi 4 Model B and the Intel D435i Depth Camera. This was one of the first steps in this project as a robot is a necessity to test each algorithm.

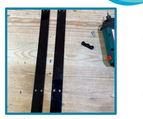


Figure 4.3. Delrin Supports
Source: Author



Figure 4.4. Full Robot
Source: Author

### Gathering Data for Automobile Safety

Getting precise and accurate data from the robot is an important step in determining the reliability each coded algorithm achieves. To get data from the robot, the functions would **send their returned data through a Micro USB cable** from the VEX Brain to the Raspberry Pi 4 Model B (Fig 5.1). This data could then be stored and transferred to a laptop where **visuals and graphs were made in QT Creator 15.0.0**. After making these visuals using the QGraphics library, logical and concise conclusions would be performed to ensure the algorithm is safe for real-world action.

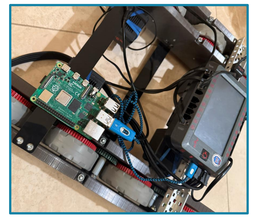


Figure 5.1. Pulling data from the robot
Source: Author

## Results

### Odometry Tracking

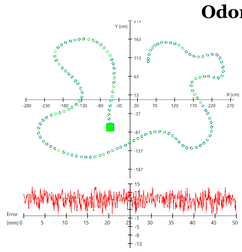



Figure 6.1. Odometry Tracking and Error Data
Source: Author

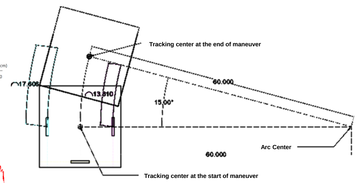



Figure 6.2. Odometry Diagram
Source: wiki.purduesigbots.com

The odometry localization algorithm (Fig 6.1) uses the rotations from the wheels to calculate the robot's pose on the field, returning a **<x, y, θ> vector** (Fig 6.2). First calculate $\Delta S$ and $\Delta\theta$ by getting data from the motor encoders then iterates a new calculation for a new pose from the odometry equations (Eqs 1.1). From testing, the odometry algorithm calculates a pose with a maximum error of **14.113mm.**

$$\Delta S = \frac{\Delta S_L + \Delta S_R}{2}$$

$$\Delta\theta = \frac{\Delta S_R - \Delta S_L}{W}$$

$$x_{\text{new}} = x + \Delta S \cos\left(\theta + \frac{\Delta\theta}{2}\right)$$

$$y_{\text{new}} = y + \Delta S \sin\left(\theta + \frac{\Delta\theta}{2}\right)$$

$$\theta_{\text{new}} = \theta + \Delta\theta$$

Equations 1.1. Odometry Equations

### Pure Pursuit Path Following

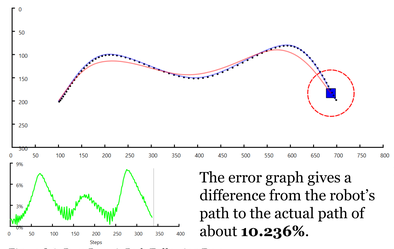



Figure 8.1. Pure Pursuit Path Following Data
Source: Author

The error graph gives a difference from the robot's path to the actual path of about **10.236%**.

The pure pursuit path following algorithm uses a **lookahead distance** to see which point the robot steers to next (Fig 8.1). Using the pure pursuit

$$\delta = \arctan\left(\frac{2\,L\,\sin(\alpha)}{L_d}\right)$$

Equation 3.1. Pure Pursuit Steering Law

a calculation of angle $\delta$ will steer the robot in proportion to the lookahead distance to the path (Eq 3.1).

### SSD Object Recognition AI Model

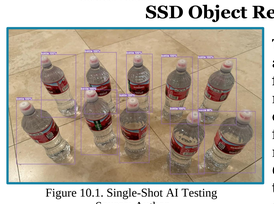


Figure 10.1. Single-Shot AI Testing
Source: Author

The SSD Object Recognition AI Model (Fig 10.1) is an approach to detecting objects using a **single frame** using the Intel D435i Depth Camera. This model is trained to determine the location of objects using **anchors**, which are **parametrized** from a bounded box to an offset in which the model is accurately able to predict the object (Eqs 6.2) . Loss functions (Eqs 6.1) are used to provide the **probability** of successfully detecting an object, where $L(\{p\}, \{t\})$ provides the total loss.

$$L_{\text{conf}} = \sum_{i \in \text{Pos}} -\ln p_{i,c_i} + \sum_{j \in \text{Neg}} -\ln p_{j,0}$$

$$L_{\text{loc}} = \sum_{i \in \text{Pos}} \sum_{\ell \in \{x,y,w,h\}} \text{SmoothL1}\left(\hat{t}_{i,\ell} - t_{i,\ell}\right)$$

$$L(\{p\},\{t\}) = \frac{1}{N}\left(L_{\text{conf}}(\{p\}) + \alpha\, L_{\text{loc}}(\{t\})\right)$$

Equations 6.1. Single-Shot AI Loss Functions

$$d_{k,ij}^{(a)} = \left(c_x,\, c_y,\, w,\, h\right), \quad a = 1, \ldots, A_k.$$

$$t_x = \frac{g_x - c_x}{w}, \quad t_y = \frac{g_y - c_y}{h}, \quad t_w = \ln\left(\frac{g_w}{w}\right), \quad t_h = \ln\left(\frac{g_h}{h}\right)$$

$$\hat{g}_x = w\,\hat{t}_x + c_x, \quad \hat{g}_y = h\,\hat{t}_y + c_y, \quad \hat{g}_w = w\,e^{(\hat{t}_w)}, \quad \hat{g}_h = h\,e^{(\hat{t}_w)}$$

Equations 6.2. Parametrization and Prediction Functions

### Dynamic Window Approach Path Planner

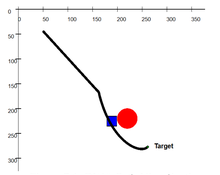



Figure 7.1. DWA Path Visualization
Source: Author

| Clear | Max V | Max ω | α | β | γ |
|---|---|---|---|---|---|
| 40cm | 5.11 ft/s | 2.98 ft/s | 0.54 | 0.01 | 0.82 |

Table 1.1. DWA Constraints and Weights
Source: Author

The DWA Path Planner (Fig 7.1) accurately avoids obstacles by predicting the robot's movements using angular and linear velocities. This algorithm then uses a **dynamic window** for states where the robot could be and assigns a value to each of these measurements using a cost function (Eqs 2.1). This cost function uses heading, velocity, and clearance to determine which point is most optimal. Furthermore, weights **α, β, γ** were added to these values to tune the planner (Table 1.1).

$$J = \alpha J_{\text{heading}} + \beta J_{\text{velocity}} + \gamma J_{\text{obstacle}}$$

$$J_{\text{heading}} = 1 - \cos(\theta_{\text{goal}} - \theta_{\text{traj}})$$

$$J_{\text{velocity}} = 1 - \frac{v}{v_{\max}}$$

$$J_{\text{obstacle}} = \frac{1}{d_{\min}}$$

$$\vec{\nabla} J_{\text{obstacle}} = -\frac{1}{d_{\min}^2}\vec{\nabla} d_{\min}$$

$$\kappa = \frac{d\theta}{ds}, \kappa \approx \frac{\omega}{v}$$

$$\arg\min_{(v,\omega)} J(v,\omega)$$

Equations 2.1. DWA Equations

### Model Predictive Control Path Following

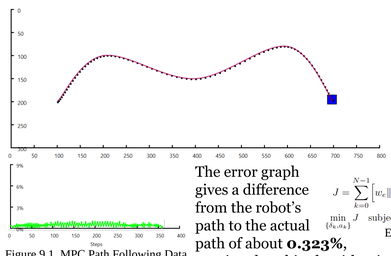



Figure 9.1. MPC Path Following Data
Source: Author

The error graph gives a difference from the robot's path to the actual path of about **0.323%**, proving that this algorithm is reliable for the safety of passengers. This value could be lower with better tuned constraints (Table 2.1).

$$-\delta_{\max} \le \delta_k \le \delta_{\max},$$
$$-a_{\max} \le a_k \le a_{\max},$$
$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k)$$

| $\delta_k$ | $a_k$ |
|---|---|
| 0.3 | 0.7 |

Table 2.1. MPC Constraints
Source: Author

The next tested path following algorithm, Model Predictive Control (MPC) (Fig 9.1), uses a **prediction horizon** $N$ that gives time steps $k$ for the robot to calculate (Eqs 4.2). Calculate the MPC cost for each step using function $J$ (Eq 4.1) then **minimize** it subject to the following time step.

$$J = \sum_{k=0}^{N-1}\left[w_c\|\mathbf{x}_k - \mathbf{x}_{\text{ref},k}\|^2 + w_u\|\mathbf{u}_k\|^2\right] + w_c\|\mathbf{x}_N - \mathbf{x}_{\text{ref},N}\|^2$$
$$\min_{\{\delta_k, a_k\}} J \quad \text{subject to} \quad \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \quad (\delta_k, a_k) \in \mathcal{U}, \quad \mathbf{x}_k \in \mathcal{X}$$

Equation 4.1. MPC Cost Function

$$x_{k+1} = x_k + v_k \cos(\theta_k)\,\Delta t$$
$$y_{k+1} = y_k + v_k \sin(\theta_k)\,\Delta t$$
$$\theta_{k+1} = \theta_k + \frac{v_k}{L}\tan(\delta_k)\,\Delta t$$
$$v_{k+1} = v_k + a_k\,\Delta t$$

Equations 4.2. MPC Time Step Parameters

### Testing All Algorithms Together

The DWA Path Planner algorithm and Single-Shot Object Detection AI was downloaded onto the Raspberry Pi 4 Model B due to **the aggressive calculations** needed. The other algorithms (MPC and Odometry) were downloaded onto the VEX Brain as data transferring the calculations to send voltage to the motors is much easier. The obstacles course consisting of 10 **randomly placed water bottles and small objects** proved a **success rate of 98%** over 50 trials (Fig 11.1).



Figure 11.1. Full test with VEX Robot
Source: Author

## Discussion

The results of this study demonstrate the effectiveness of integrating Model Predictive Control (MPC), Dynamic Window Approach (DWA) path planning, and Single-Shot Detection (SSD) AI object recognition in improving the safety and precision of autonomous vehicle navigation. The data collected from 50 obstacle course trials revealed a **98% success rate** in avoiding collisions, highlighting the system's capability to run in real-world scenarios. The MPC path-following algorithm exhibited an impressively low path deviation of **0.323%** significantly outperforming the pure pursuit algorithm, which had a deviation of **10.236%**. The significance of these results are highlighted in comparison to previous studies (Fig 12.1) on reactive systems. On robots of similar size and tasks, a reactive planning system **struggled in complex environments** due to its reliance on immediate sensor data, resulting in suboptimal path tracking and increased likelihood of collisions (Fig 12.2).

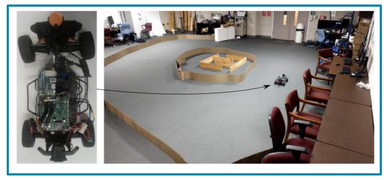


Figure 12.1. Karimi Research Experiment Using MPC
Source: Karimi, 2021

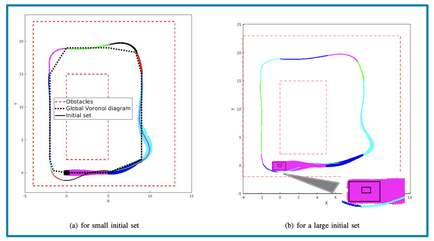



Figure 12.2. Karimi Research Results Using MPC
Source: Karimi, 2021

The hypothesis that integrating predictive control with advanced AI and path-planning algorithms would enhance vehicle safety and precision was strongly supported by the collected data. Compared to reactive models, which rely on instantaneous sensor readings and struggle with long-term planning, this predictive approach **ensures more reliable and stable vehicle behavior**.

The effectiveness of the DWA path planner further strengthens the model's advantage, as it optimizes obstacle avoidance by **assessing multiple possible trajectories** and selecting the safest option based on a cost function. Additionally, the SSD AI object recognition model significantly **enhances real-time responsiveness** by detecting and classifying obstacles on a faster and more accurate level. This makes it particularly **useful in unpredictable environments** where obstacles may suddenly appear, as it provides a balanced trade-off between aggressive maneuvering and maintaining a stable trajectory. The harmony between the DWA and the MPC enhances the overall system performance, as the DWA determines the best path around obstacles and the MPC ensures that the vehicle follows this path with **minimal deviation and optimal control over movement**.

In conclusion, this study provides strong evidence that predictive control models, particularly MPS, **outperform traditional reactive approaches in autonomous navigation systems**. The extreme reduction in path deviation and significantly higher success rate in obstacle avoidance validate the superiority of predictive models. As autonomous vehicle technology continues to advance, incorporating predictive control is **essential in achieving safer and more reliable vehicle systems**.

## Future Implications

This project is far from finished as some issues need to be fixed and redesigned, such as the SSD AI which **cannot detect extremely small objects**. Furthermore, other algorithms for path planning and path following can be tested, such as a **PID controller** and **A* path planning**. This project aims to protect the safety of transportation and believes that **every life matters**. Therefore, even if one error occurs, it needs to be fixed, reiterated, and tested to guarantee a **risk-free technology** that anyone can use. The path planning technology would also need to be implicated into actual self-driving cars for further testing. In the future, this project **plans to upgrade current systems and test new methods for the robot:**

- **Develop a new AI model (Faster R-CNN)** that is trained using more layers and images, increasing the reliability for object recognition. This new AI method, using a two-step prediction method, will help ensure more probable results
- Upgrade the robot to a **faster drive speed** or to a **larger model** such as an **FRC robot**, making sure this technology is precise at life-threatening speeds and sizes, providing a safe environment in an event of an accident
- Code and test new pathing algorithms such as the **A* path planner** (Fig 13.1) and the **Proportional Integral Derivative (PID) path following controller** (Fig 13.2) which could prove to be more accurate and faster at calculations

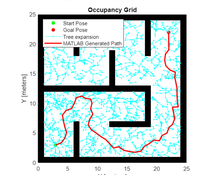



Figure 13.1. A* Path Planner Example
Source: mathworks.com

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{d}{dt}e(t)$$

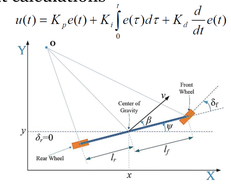



Figure 13.2. PID Path Following Example
Source: Farag, 2020

This project plans to continue this extraordinary research, focusing on an implementation for more advanced techniques. As more autonomous automobiles enter the roads, this technology will prove to be useful in **saving lives** daily.